

Data and Artificial Intelligence

Cyber Shujaa Program

Week 11 Assignment

Natural Language Processing with

Transformers

Student Name: Violet Joy

Student ID: cs-da01-25025

Table of Content

Data and Artificial Intelligence	1
Cyber Shujaa Program	1
Week 11 Assignment Natural Language Processing with Transformers	1
Introduction.....	3
Tasks Completed	4
2.1: Load pre-trained.....	4
2.2: Dataset Preparation: Original and Custom Sentence Pair Generation.....	5
2.3: Extracting Sentence-Level Representations Using the BERT [CLS] Token	6
2.4: Defining a Similarity Threshold for Sentence Pair Classification.....	8
2.5: Evaluating Prediction Accuracy Against Ground Truth Labels.....	9
2.6: Summary.....	10
Link to Code:	11
Conclusion	12

Introduction

In this assignment, I explore key concepts of Natural Language Processing (NLP) by leveraging a pre-trained BERT (Bidirectional Encoder Representations from Transformers) model from Hugging Face Transformers. Specifically, I focus on sentence encoding, contextual word embeddings, and semantic similarity.

Using BERT, I extract token-level embeddings and compute sentence similarity based on the cosine similarity of [CLS] token representations. This practical exercise highlights BERT's ability to capture word meaning based on context a critical advantage over traditional NLP methods such as Bag of Words and TF-IDF which rely on static, context independent representations.

Through analyzing ten sentence pairs (five provided and five created), I demonstrate how contextual embeddings enable deeper semantic understanding, especially in cases of polysemy (words with multiple meanings). By applying a similarity threshold and evaluating my predictions against manually assigned labels, I also assess the effectiveness of BERT in identifying semantically similar sentences.

This assignment deepens my understanding of BERT's encoder architecture, the role of contextualized embeddings and the application of cosine similarity in real-world NLP tasks.

Tasks Completed

2.1: Load pre-trained BERT tokenizer and BERT model from Hugging Face

In this step, I load a pre-trained BERT model and its associated tokenizer using the Hugging Face transformers library

Purpose of the Tokenizer

The tokenizer is responsible for:

- **Breaking down (tokenizing)** input sentences into sub word units (Word Piece tokens).
- **Mapping tokens to numerical IDs** using the model's vocabulary.
- **Adding special tokens** like [CLS] used to represent the whole sentence and [SEP] used to separate two sentences.
- **Padding or truncating** inputs to a consistent length for batching.
- **Creating attention masks** to indicate which tokens should be attended to and which should be ignored (padding).

Purpose of the Pre-trained BERT Model

The model itself usually bert-base-uncased has already been trained on a large corpus like Wikipedia and Book Corpus and can:

- Convert input token IDs into high-dimensional embeddings.
- Capture deep contextual relationships between words.
- Provide token-level and sentence-level representations, especially via the [CLS] token for classification and similarity tasks.

```

NL with transformers.ipynb
File Edit View Insert Runtime Tools Help

Load pre-trained BERT tokenizer and BERT model from Hugging Face

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = TFBertModel.from_pretrained('bert-base-uncased')

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 915B/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 1.70MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 4.70MB/s]
config.json: 100% 570/570 [00:00<00:00, 17.4kB/s]
model.safetensors: 100% 440M/440M [00:07<00:00, 86.8MB/s]

TensorFlow and JAX classes are deprecated and will be removed in Transformers v5. We recommend migrating to PyTorch classes or pinning

```

Fig 2.1.1: Output of loading pre-trained BERT

2.2: Dataset Preparation: Original and Custom Sentence Pair Generation

To evaluate the effectiveness of BERT based contextual embeddings in capturing sentence level semantic similarity, a dataset consisting of 10 sentence pairs was prepared. This dataset was divided into two parts: five sentence pairs provided as part of the assignment and five additional sentence pairs manually created to extend the evaluation scope.

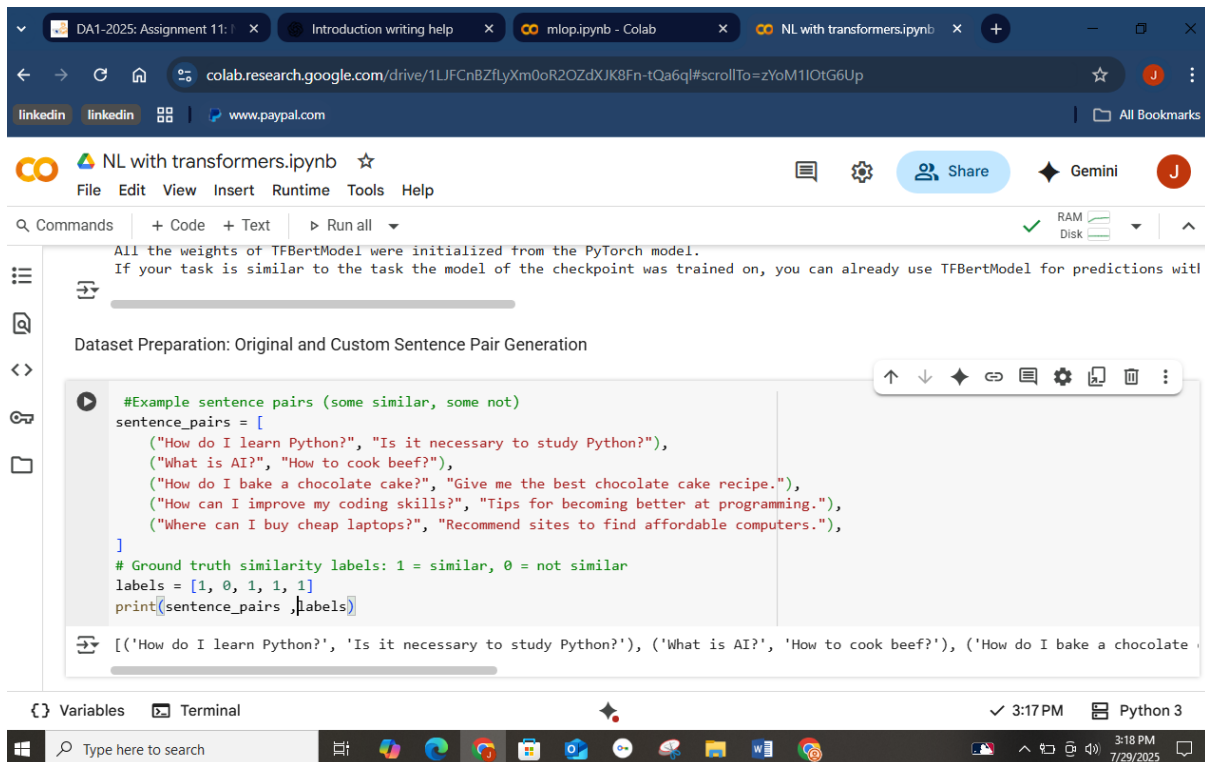
The original sentence pairs represent a controlled sample meant to test common cases of sentence similarity and dissimilarity. To complement this, the custom sentence pairs are carefully designed to introduce a broader range of linguistic variation. These include examples with:

- Synonyms and paraphrased content to test semantic similarity,
- Polysemous words in different contexts to assess BERT's contextual sensitivity,
- Sentence pairs with similar structure but different meanings to challenge surface-level similarity.

Each pair was manually labeled with a ground truth value:

- **1** for semantically similar sentences,
- **0** for sentences that differ in meaning.

This dataset serves as the foundation for subsequent embedding extraction, cosine similarity computation and accuracy evaluation. By incorporating both provided and custom examples, the dataset offers a balanced test bed for assessing BERT's capability in distinguishing nuanced semantic relationships.



The screenshot shows a Google Colab notebook titled "NL with transformers.ipynb". The notebook is open to a cell containing Python code that defines a list of sentence pairs and their corresponding ground truth similarity labels. The code is as follows:

```
#Example sentence pairs (some similar, some not)
sentence_pairs = [
    ("How do I learn Python?", "Is it necessary to study Python?"),
    ("What is AI?", "How to cook beef?"),
    ("How do I bake a chocolate cake?", "Give me the best chocolate cake recipe."),
    ("How can I improve my coding skills?", "Tips for becoming better at programming."),
    ("Where can I buy cheap laptops?", "Recommend sites to find affordable computers."),
]

# Ground truth similarity labels: 1 = similar, 0 = not similar
labels = [1, 0, 1, 1, 1]
print(sentence_pairs, labels)
```

The output of the code is displayed below the cell, showing the sentence pairs and their corresponding labels as a list of tuples:

```
[('How do I learn Python?', 'Is it necessary to study Python?'), ('What is AI?', 'How to cook beef?'), ('How do I bake a chocolate cake?', 'Give me the best chocolate cake recipe.'), ('How can I improve my coding skills?', 'Tips for becoming better at programming.'), ('Where can I buy cheap laptops?', 'Recommend sites to find affordable computers.'), [1, 0, 1, 1, 1]]
```

Fig 2.2.1: Output of sentence_pairs and labels

2.3: Extracting Sentence-Level Representations Using the BERT [CLS] Token

To compare sentence meanings and compute semantic similarity, it is necessary to have a numerical representation of each sentence. BERT provides a powerful way to achieve this through its contextual embeddings. Specifically, I use the embedding associated with the special [CLS] classification token to represent the entire sentence.

The [CLS] token is a special token added to the beginning of every input sequence in BERT. During training, BERT learns to use the final hidden state of this token as an aggregate representation of the entire sentence or sentence pair. This makes it especially useful for tasks like classification, similarity analysis and sentence embedding.

Why Use the [CLS] Token for Sentence Representation?

- It provides a **single, fixed-length vector** for any input sentence, regardless of its length.
- It encodes **contextual information** from the entire sequence due to BERT's bidirectional attention mechanism.
- It is widely adopted in sentence-level tasks like sentence similarity, paraphrase detection, and semantic search.

Process of Extracting [CLS] Embeddings

Tokenization:

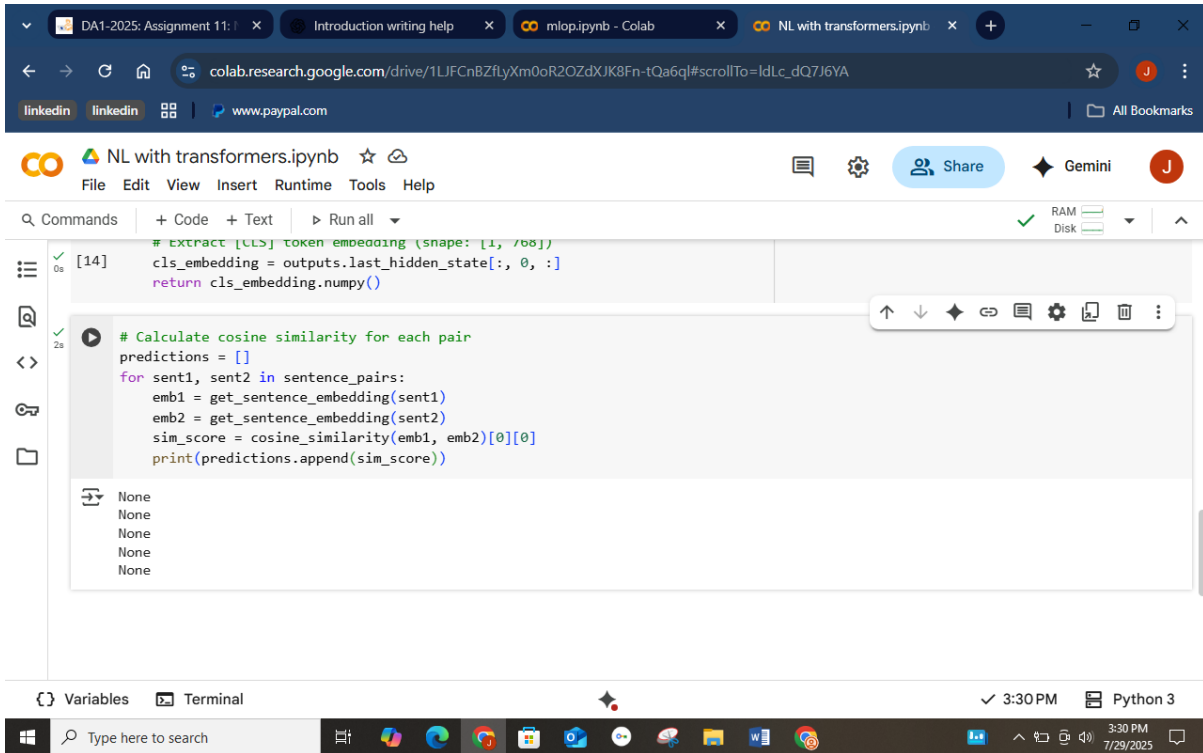
The sentence is tokenized using the BERT tokenizer, which includes adding special tokens like [CLS] and [SEP] and converting words into token IDs.

Embedding Generation:

The input token IDs are passed through the BERT model, which outputs hidden states for each token.

[CLS] Vector Extraction:

The final hidden state corresponding to the [CLS] token is extracted and used as the sentence-level embedding.



DA1-2025: Assignment 11: | Introduction writing help | mlop.ipynb - Colab | NL with transformers.ipynb |

colab.research.google.com/drive/1LJFCnBZflyXm0oR2OZdXJK8Fn-tQa6ql#scrollTo=IdLc_dQ7J6YA

linkedin | www.paypal.com

NL with transformers.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[14] # Extract [CLS] token embedding (shape: [1, 768])
      cls_embedding = outputs.last_hidden_state[:, 0, :]
      return cls_embedding.numpy()

# Calculate cosine similarity for each pair
predictions = []
for sent1, sent2 in sentence_pairs:
    emb1 = get_sentence_embedding(sent1)
    emb2 = get_sentence_embedding(sent2)
    sim_score = cosine_similarity(emb1, emb2)[0][0]
    print(predictions.append(sim_score))
```

None
None
None
None
None

Variables Terminal

3:30 PM Python 3

Type here to search

3:30 PM 7/29/2025

Fig 2.3.1:Output of calculating cosine similarity for each pair

2.4: Defining a Similarity Threshold for Sentence Pair Classification

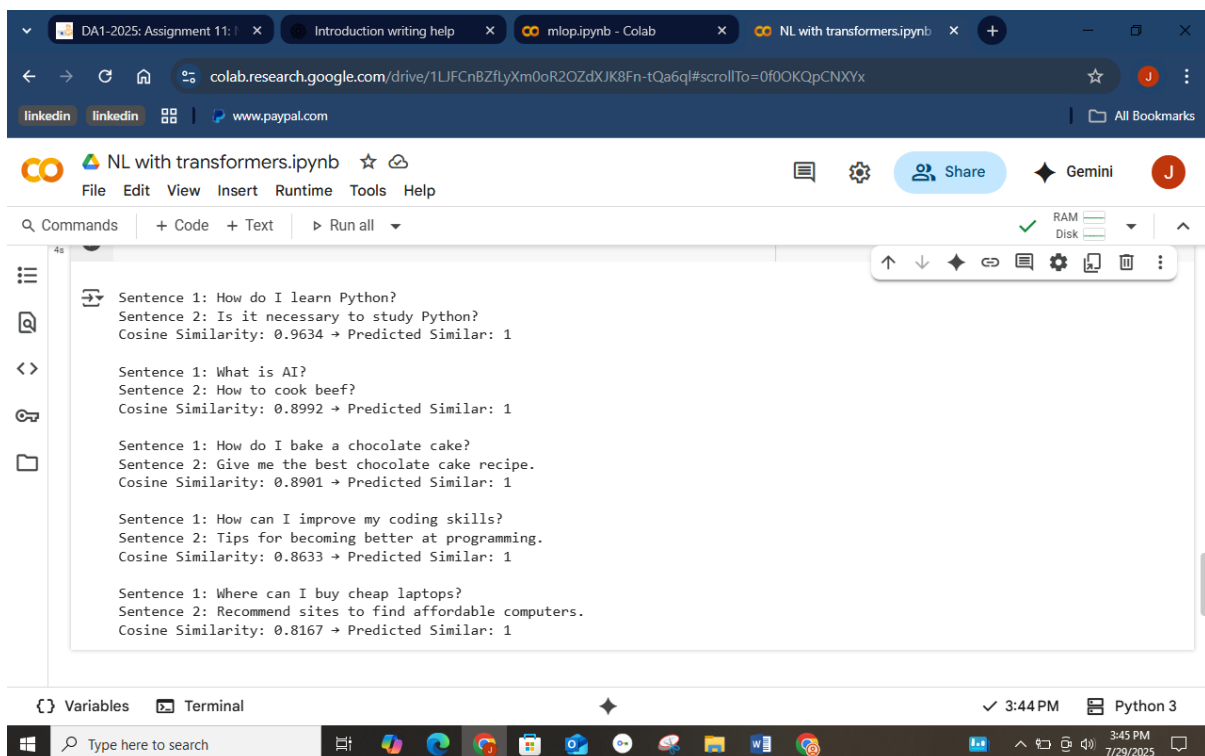
Once sentence embeddings are extracted using BERT, the next step in semantic similarity analysis is to determine whether two sentences convey similar meaning. To achieve this, I used **cosine similarity** a metric that quantifies the angular similarity between two vectors in a high dimensional space.

In this assignment, I define a **threshold of 0.7** to distinguish between similar and dissimilar sentence pairs. The rule is as follows:

- If **cosine similarity** > 0.7 , the sentence pair is considered **semantically similar** and assigned a label of **1**.
- If **cosine similarity** ≤ 0.7 , the pair is considered **not similar** and labeled as **0**.

The threshold acts as a **decision boundary** in this binary classification task. It provides a simple yet effective way to convert continuous similarity scores into discrete class labels. A threshold of 0.7 is commonly used in semantic similarity tasks involving contextual embeddings, as it tends to reflect a reasonably high degree of semantic overlap without being overly strict. However, the choice of threshold can be adjusted based on:

- The nature of the dataset,
- The granularity of similarity you want to capture,
- Empirical performance (e.g., maximizing accuracy, precision, or recall).



```

Sentence 1: How do I learn Python?
Sentence 2: Is it necessary to study Python?
Cosine Similarity: 0.9634 → Predicted Similar: 1

Sentence 1: What is AI?
Sentence 2: How to cook beef?
Cosine Similarity: 0.8992 → Predicted Similar: 1

Sentence 1: How do I bake a chocolate cake?
Sentence 2: Give me the best chocolate cake recipe.
Cosine Similarity: 0.8901 → Predicted Similar: 1

Sentence 1: How can I improve my coding skills?
Sentence 2: Tips for becoming better at programming.
Cosine Similarity: 0.8633 → Predicted Similar: 1

Sentence 1: Where can I buy cheap laptops?
Sentence 2: Recommend sites to find affordable computers.
Cosine Similarity: 0.8167 → Predicted Similar: 1

```

Fig 2.4.1: Output of predictions on sentences

2.5: Evaluating Prediction Accuracy Against Ground Truth Labels

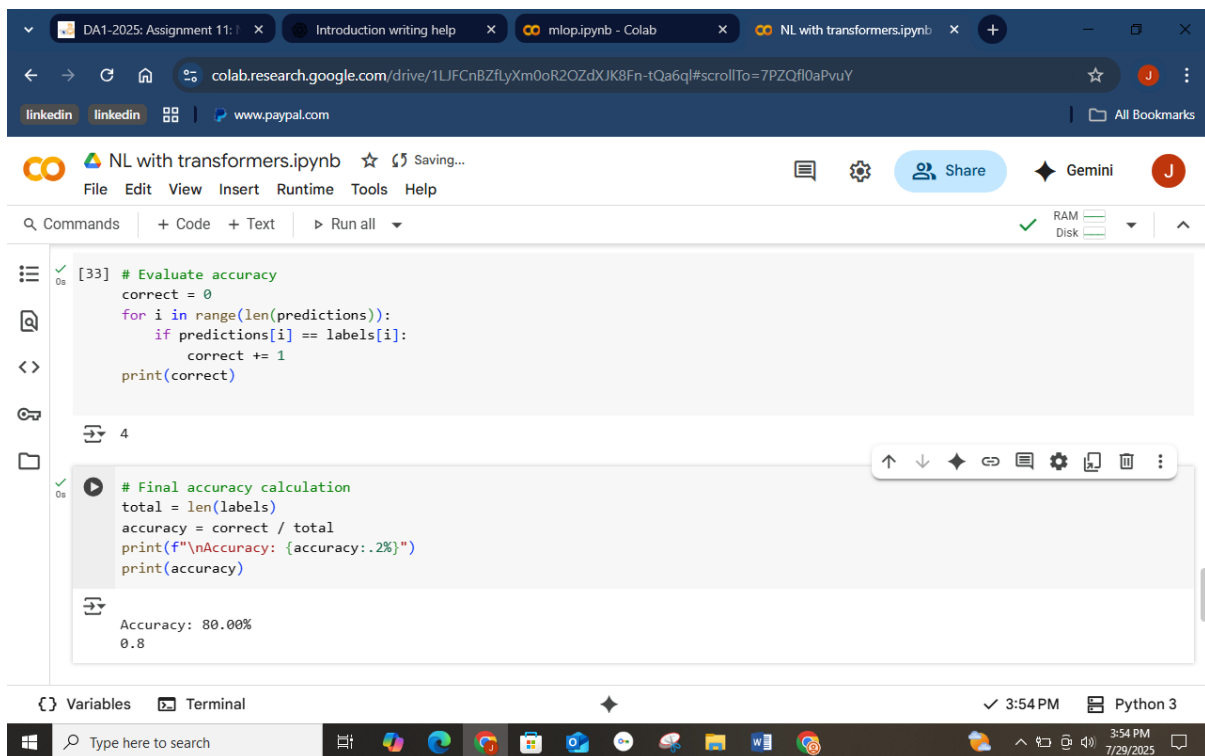
After computing cosine similarity scores and applying the 0.7 threshold to generate predicted similarity labels for each sentence pair, the next step is to evaluate the performance of the model. This is done by comparing the predicted labels with manually assigned **ground truth labels**, where:

- **1** indicates the sentence pair is semantically similar,
- **0** indicates the pair is not similar.

To measure how well the model's predictions align with human judgment, we calculate **accuracy**, defined as:

Accuracy = Total Number of Predictions / Number of Correct Predictions

This metric provides a simple yet effective evaluation of the model's performance in distinguishing between semantically similar and dissimilar sentence pairs. A high accuracy score would indicate that BERT's sentence embeddings, combined with cosine similarity, effectively capture the semantic relationships in the dataset.



```
[33] # Evaluate accuracy
correct = 0
for i in range(len(predictions)):
    if predictions[i] == labels[i]:
        correct += 1
print(correct)

4

# Final accuracy calculation
total = len(labels)
accuracy = correct / total
print(f"\nAccuracy: {accuracy:.2%}")
print(accuracy)

Accuracy: 80.00%
0.8
```

Fig 2.5.1: Output of evaluating accuracy

2.6: Summary

BERT differs significantly from traditional NLP approaches such as Bag of Words and TF-IDF. While traditional methods rely on static word representations and ignore the context in which words appear, BERT generates **contextual embeddings** meaning each word's vector representation is influenced by the words around it. This allows BERT to capture nuanced meanings, such as distinguishing between polysemous words based on context. At the core of BERT is its **encoder**, a stack of Transformer layers that process input sequences bi-directionally. The encoder plays a crucial role in this assignment by producing deep, context-aware token embeddings for each sentence.

Among these tokens, the special **[CLS] token**, inserted at the beginning of every input, is used to derive a single, fixed-length embedding that summarizes the entire sentence. This embedding is extracted from the final layer of the model and serves as the basis for comparing sentence meaning. To measure semantic similarity between two sentences, the assignment uses **cosine similarity**, a metric that assesses the angular distance between two vectors in embedding space. A higher cosine similarity indicates greater semantic overlap. Together, these components enable BERT to perform sentence similarity classification with a much deeper understanding of language than traditional NLP techniques.

Link to Code:

<https://colab.research.google.com/drive/1LJFCnBZfLyXm0oR2OZdXJK8Fn-tQa6ql?usp=sharing>

Conclusion

This assignment provided a hands-on understanding of how modern NLP models like BERT revolutionize text representation through contextual embeddings. By encoding sentences using a pre-trained BERT model, I was able to see firsthand how the meaning of words shifts based on context something traditional approaches like Bag of Words or TF-IDF fail to capture.

Using the [CLS] token embeddings and computing cosine similarity allowed me to classify sentence pairs as similar or not and compare the results with manually assigned ground truth labels. This process not only helped me better understand semantic similarity but also showed the strengths and limitations of using a fixed similarity threshold.

Through this exercise, I deepened my understanding of how BERT's encoder functions to generate meaningful, context aware embeddings. One of the most insightful aspects was observing how polysemous words are handled differently depending on their context a clear demonstration of BERT's power.

Working on this assignment challenged me to move beyond theoretical knowledge and apply core NLP concepts in code. I gained practical experience with the Hugging Face Transformers library and developed a better appreciation for how deep learning models process language. It also helped me become more confident in evaluating model performance and understanding how embedding-based similarity works in real-world scenarios. Overall, this task significantly strengthened my understanding of contextual language models and their practical applications in NLP.